Question 1: Story and MVP

1. Roles, relationships, and layered dependencies

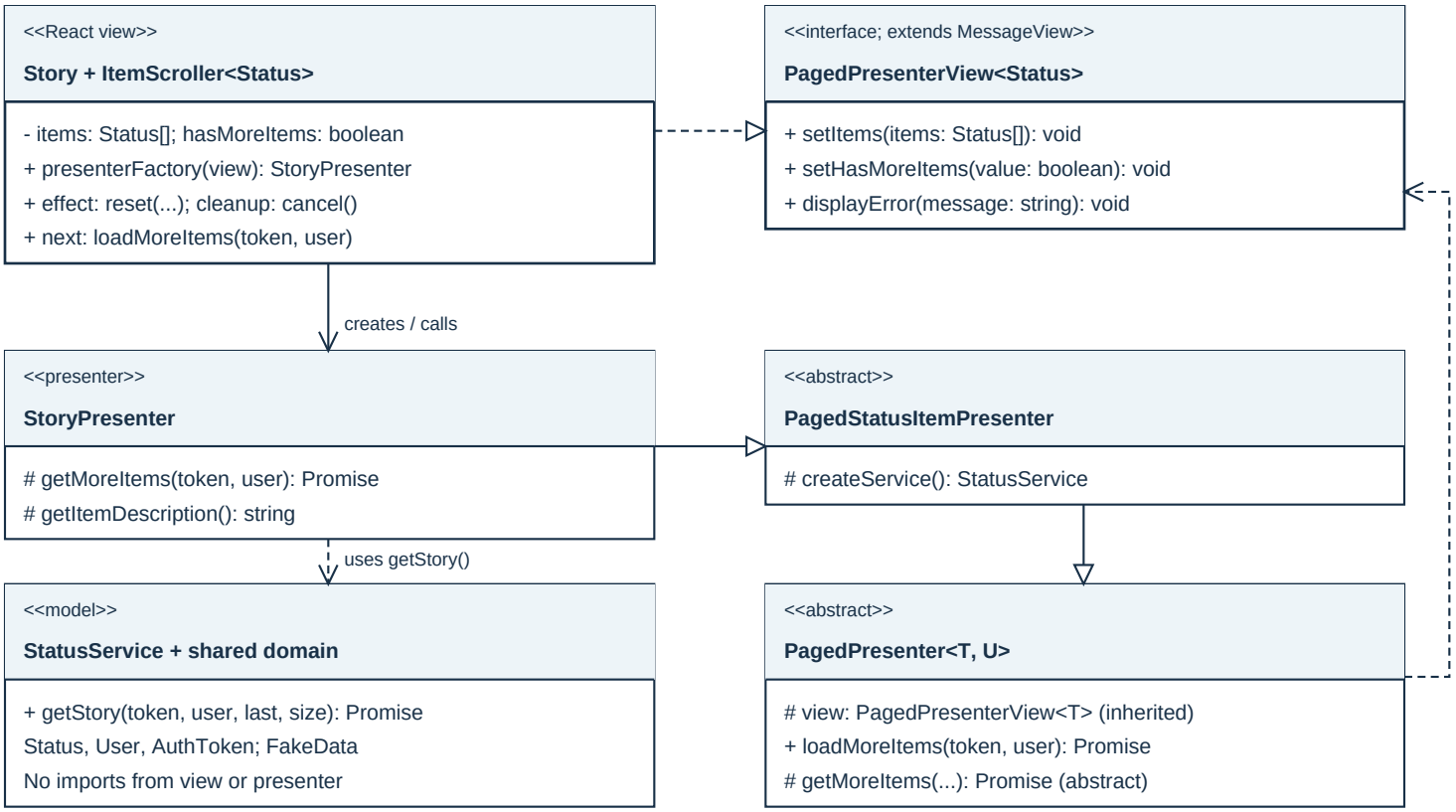
The view consists of the Story React component and its ItemScroller<Status> child. Story supplies the StoryPresenter factory and renders each status through StatusItem. ItemScroller owns React state for items and hasMoreItems; it supplies a PagedPresenterView<Status> object whose setItems, setHasMoreItems, and displayError callbacks update state or display an error toast.

StoryPresenter is the concrete presenter. It inherits the paging algorithm from PagedPresenter<Status, StatusService> through PagedStatusItemPresenter. Its getMoreItems override calls StatusService.getStory(token, user, lastItem, pageSize), and its getItemDescription override supplies "story" for error messages. The inherited pageSize is 10.

The model includes StatusService and the shared Status, User, AuthToken, and FakeData types. StatusService.getStory currently delegates to getFeed, which returns a page of hardcoded statuses from FakeData. There is no network request or persistent story update at this milestone.

The view depends on presenter types and constructs StoryPresenter through a factory. The presenter depends on the PagedPresenterView interface, not on React, ItemScroller, StatusItem, or the toast implementation. PagedPresenterView extends MessageView for displayError. The view supplies the implementation of that interface, so callbacks cross back into the UI without a presenter importing the UI layer. The model imports no view or presenter. Shared domain types can be used across layers. These source dependencies preserve the UI-to-presenter-to-model direction; the diagram's callback arrows represent runtime calls, not reverse imports.

Figure 1. Story class and component relationships



Hollow triangles: inheritance / interface realization. Dashed open arrows: dependencies.

Question 1: Story interaction

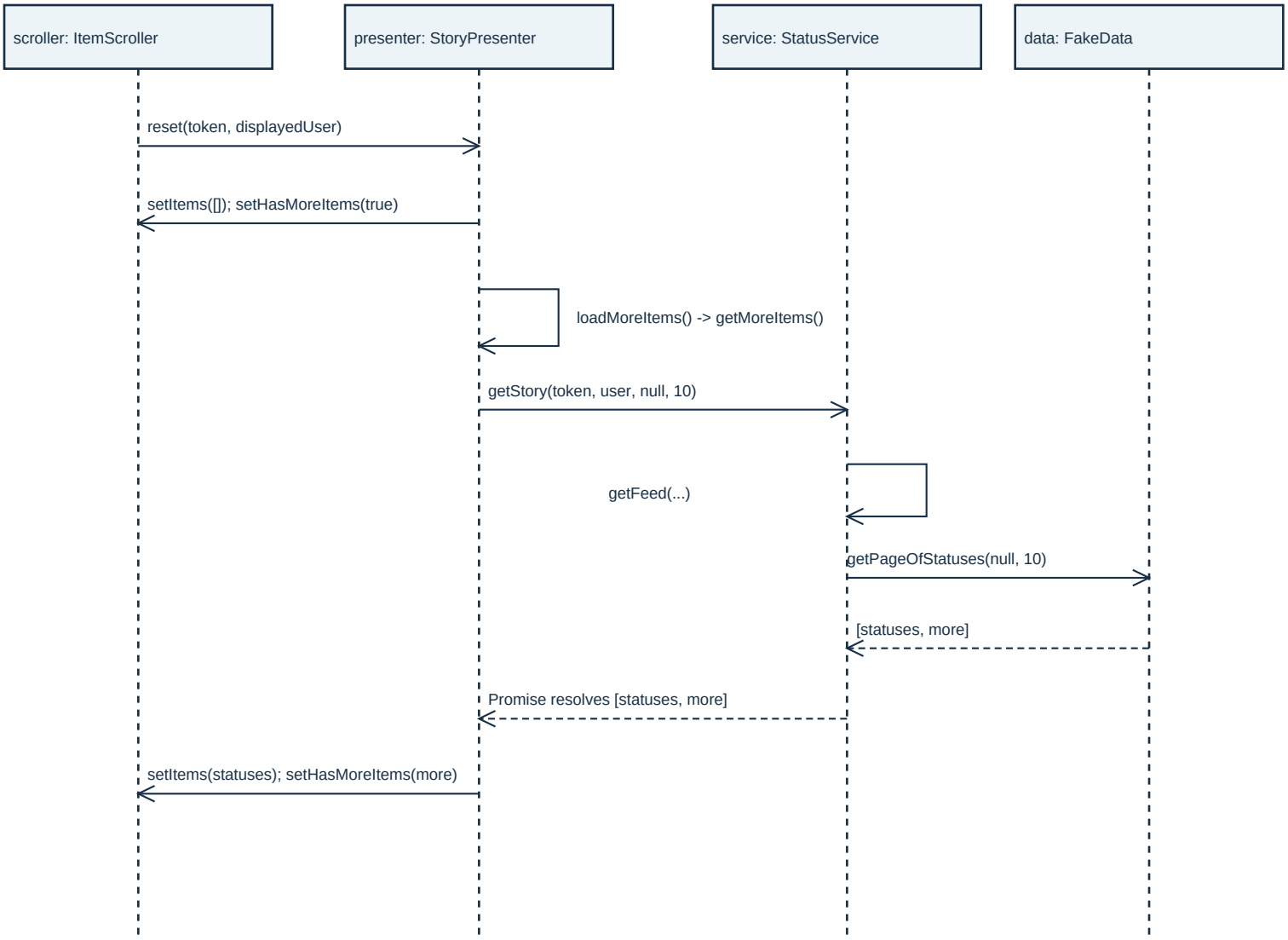
2. Why MVP is effective, with a successful asynchronous page request

1. Separation of responsibilities: ItemScroller renders state, StoryPresenter chooses the story operation, and StatusService supplies dummy model data. Replacing dummy data with a backend can happen in the service without moving paging logic into React.

2. Testability: presenters can receive a fake view and service without mounting React. View tests can substitute a presenter. The milestone's navbar and posting tests use these same boundaries.

3. Reuse and consistent behavior: StoryPresenter and FeedPresenter share paging, error handling, cancellation, and concurrency guards while selecting different service methods. The same ItemScroller renders either list.

Figure 2. First story page: sequence diagram



The presenter awaits the service Promise. Solid arrows are calls; dashed arrows are returned results. loadMoreItems and getMoreItems execute on the same StoryPresenter object through inherited methods. React re-renders the rows using StatusItem. Later scroll requests reuse the updated cursor.

Question 2: Template Method

A. Roles and collaboration / B. Reduction of duplication

A. The superclass fixes the algorithm; subclasses select the data

PagedPresenter<T, U> is the abstract class. Its loadMoreItems method is the template method: it fixes the ordering of the guard checks, request tracking, call to getMoreItems, stale-result check, page accumulation, cursor update, view updates, and loading cleanup. It uses inherited doFailureReportingOperation for the shared try/catch, with an onFailure callback that stops further paging. reset and cancel manage the generation counter and list lifecycle.

getMoreItems(token, user) and getItemDescription() are abstract hooks. createService() is another abstract hook used by the lazy service getter. PagedStatusItemPresenter provides the StatusService factory; StoryPresenter and FeedPresenter provide the concrete service calls and descriptions. StoryPresenter calls getStory, FeedPresenter calls getFeed, and both use the inherited lastItem and pageSize. PagedUserItemPresenter similarly supplies FollowService for the follower and followee presenters.

B. One implementation of the common paging steps

Without this shared algorithm, separate story and feed loaders would each need items, last, more, loading, generation, page size, guards, error handling, appending, and view updates. PagedPresenter stores and implements these once. The concrete presenters retain only their different service call and descriptive label. A correction to stale-result handling or cursor advancement therefore applies to both lists, as well as follower/followee pagination.

The PDF's "before" diagram is a conceptual reconstruction of duplicated loaders before extracting the shared algorithm, not a claim that those exact standalone classes appear in git history. The repository already used a callback-based shared ScrollerPresenter before the current hook-based hierarchy (for example, commit cc6e151). That earlier implementation already removed paging duplication; the current template-method design preserves that reuse and expresses the variation through protected overrides rather than bound callback constructor arguments. The "after" diagram reflects the current source, with only members relevant to the explanation shown.

Source map

View: tweeter-web/src/components/mainLayout/Story.tsx, ItemScroller.tsx; components/statusItem/StatusItem.tsx.

Presenter: tweeter-web/src/presenter/StoryPresenter.ts, FeedPresenter.ts, PagedStatusItemPresenter.ts, PagedPresenter.ts, Presenter.ts.

Model: tweeter-web/src/model/service/StatusService.ts; tweeter-shared/src/model/domain and util/FakeData.ts.

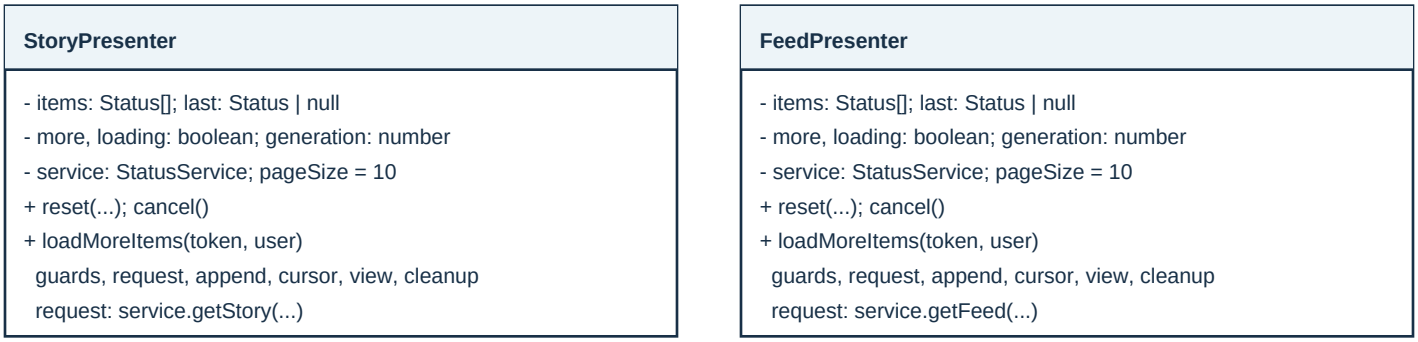
Scope of the diagrams

Only members needed for the pattern discussion are shown. Presenter stores the typed view; ErrorReportingPresenter supplies the reusable failure wrapper. Figure 4 abbreviates these inherited details so the template and its protected hooks remain visible.

Question 2: Before and after

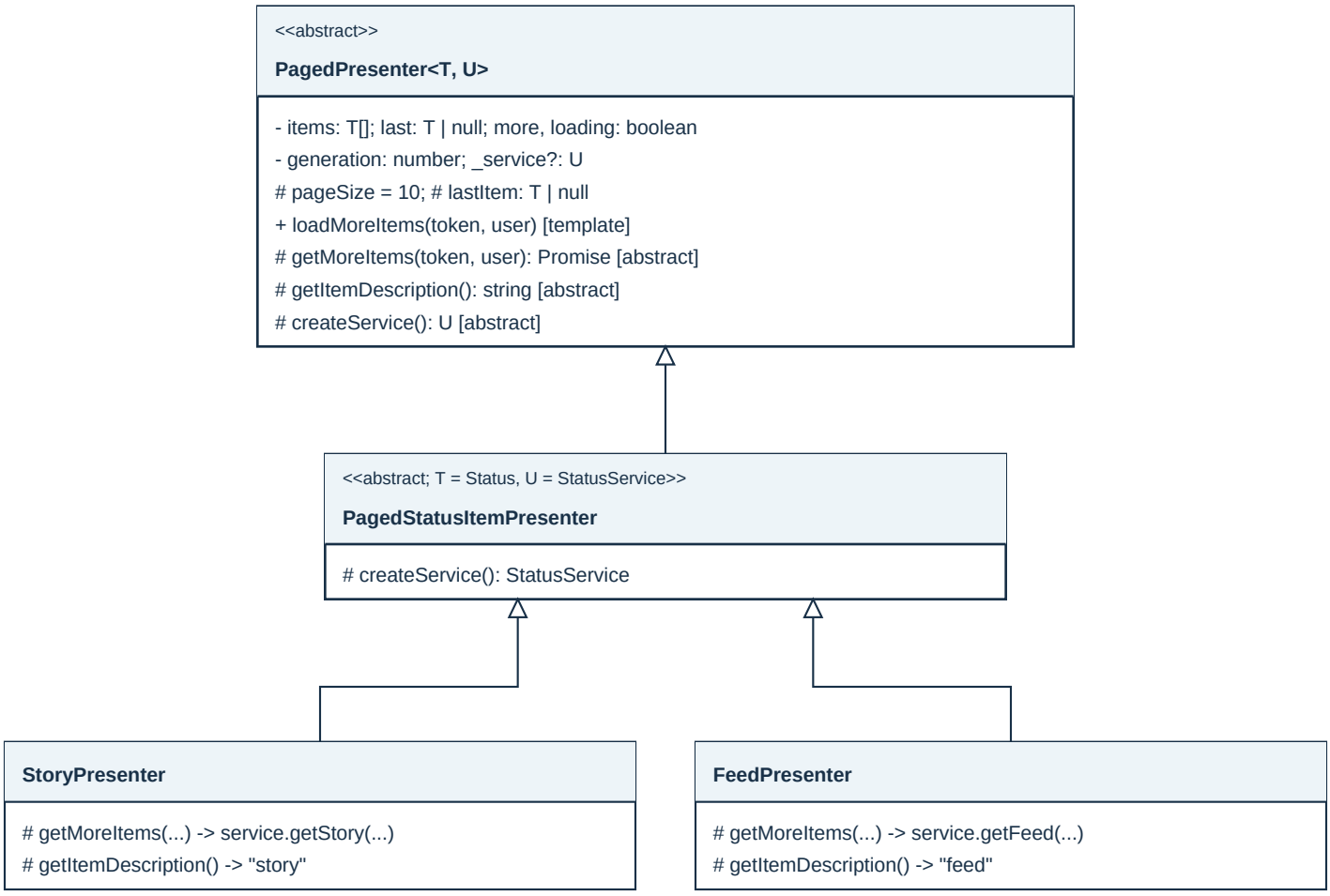
Conceptual duplicated loaders and the implemented template-method hierarchy

Figure 3. Before extraction (conceptual reconstruction)



The duplicated state and loading steps illustrate the extraction. They are not a historical snapshot; the earlier repository design already shared these steps through callback-based ScrollerPresenter.

Figure 4. After extraction (current source)



Hollow triangles point to the superclass. Protected (#) hooks vary the fixed template (+).